

ALARMS

Alarms are mechanisms that allow users to receive real-time notifications about specific events or conditions related to their IoT devices. These alarms are essential for proactive management.

Alarms in Thinger.io serve several important purposes:

1. **Real-Time Monitoring:** This feature enables users to monitor their devices in real-time and receive immediate notifications about critical events, such as device failures or abnormal conditions.
2. **Predictive Maintenance:** They help anticipate problems before they occur, which is essential for predictive maintenance. For example, receiving an alarm when a motor's temperature exceeds a specific threshold can prevent major failures.
3. **Security and Control:** They enhance security by alerting users about unauthorized access or suspicious activities. Alarms can also be used for control, activating other devices or services in response to an event.
4. **Operational Optimization:** They facilitate operational optimization by providing information on device performance, allowing for continuous adjustments and improvements in processes.
5. **Customized Notifications:** Users can define custom alarms according to their specific needs, ensuring that relevant information is delivered to the right people at the right time.

Alarm Console

To access the alarm functionality, click on the "Alarms" tab in the main menu. This will lead to the alarm's terminal, where all currently active alarm events can be viewed. This interface provides useful information related to each alarm, including the time of triggering, severity, and current status. Details about the affected device and any relevant data points that caused the alarm to be triggered are also visible.



This comprehensive overview allows for the real-time monitoring of device status and a prompt response to any issues. But if more detail is required, there is also an "Alarms Inspector" accessible using the top-right corner menu, which allows for review of the raw data of the server events that are being produced by each alarm rule.

Creating new Alarms

Alarms are generated by the server when the criteria specified in a rule are met. Users can define these alarms by accessing the "Rules" section of the alarm console.

Alarms > rules

InstancesRules

+ Add Rule

Refresh

Search Rule

☐	Rule	Description	Execution	Duration	Checks	Activations	Normalizations	Skipped
☐	Humidity_ClimaStick humedad_alberto	Alarm to monitor when the ClimaStick humidity surpasses 20%.	5d	7ms	1	0	1	0
☐	AlarmTemperature AlarmaTemp	This alarm activates when the temperature exceeds 35°C.	27s	5ms	1	1	0	0
☐	AlarmHumidity AlarmaHumedad	This alarm triggers if the humidity detected by the sensor falls below 10%.	40s	7ms	1	1	0	0

Showing 3 rules

In this section, find a list of existing rules, which allow for their management, as well as the option to create new rules by clicking the "+ Add Rule" button:

Rule Details

Rule Identifier ⓘ

highTemperature

Name ⓘ

High Temperature

Description ⓘ

Enter description

⚙️ Configuration

Enabled ⓘ

☒

Severity ⓘ

Medium

Check Interval ⓘ

30

minutes ▾

📄 Data Source

🔔 Activation

✅ Normalization

📅 Reminder

● Data1

Name ⓘ

Data1

Data Source ⓘ

From Data Bucket

Data Bucket ⓘ

Select Bucket

📄 Environmental data

+ Add Source

📄 Clone Source

🗑 Remove Source

Rule information

- **Rule Identifier:** A unique identifier without spaces and special characters must be used
- **Name:** Mnemonic identification for users
- **Description:** Additional information to help recognize the rule in the long term

Configuration

- **Enabled:** Allows enabling or disabling the execution of the rule
- **Severity:** This is used to categorize and prioritize alarms based on their importance or impact. It helps users manage and respond to different types of events more effectively.

- **Check interval:** Is used to define how frequently the system evaluates the conditions set in a rule to determine if an alarm should be triggered. It allows users to adjust the check interval based on the specific needs of their monitoring setup.

ⓘ Note that setting an appropriate check interval helps balance the frequency of evaluations with system resource usage. In scenarios requiring high responsiveness, a shorter interval may be chosen, while less critical scenarios may use a longer interval. If the interval is too long, critical issues might not be detected promptly, while a very short interval may lead to excessive checks and potential performance impacts.

Rule definition

Este submenú permite especificar el comportamiento de la regla, especificando qué variables se van a monitorizar, cual es el valor de consigna y qué resultado tendrá cuando se produzcan evaluaciones positivas de la condición. También se especifica el comportamiento de desactivación de la regla.

Data sources

This section allows users to specify which data sources will be evaluated by the rule. Data sources may include data from buckets, device properties, or the device status (allowing also to monitor device disconnections).

The screenshot shows the 'Data Source' configuration interface. At the top, there are four tabs: 'Data Source' (selected), 'Activation', 'Normalization', and 'Reminder'. Below the tabs, there are two sub-tabs: 'Data1' (selected) and 'Data2'. The 'Data1' sub-tab contains three fields: 'Name' (set to 'Data1'), 'Data Source' (set to 'From Data Bucket'), and 'Data Bucket' (set to 'Environmental data'). At the bottom, there are three buttons: '+ Add Source', 'Clone Source', and 'Remove Source'.

- **Name:** Use a unique identifier, avoiding spaces or special characters. This name will be used to identify the JSON that contains the data point.
- **Data source:** Note that depending on the data source type, the menu may expose filtering options that allow working with multiple devices with the same behavior, so just one rule can be used to monitor multiple devices. There are multiple options:
 - From Device Resource: This option requires the device to be connected in real-time. The widget will display data as it is received from the device. It's important to note that, when fed directly from a device resource, the widget will not retain the information if the dashboard is closed or refreshed, as it only displays live data from the device to the dashboard.
 - From Data Bucket: With this option, the widget takes information from a Data Bucket previously configured in the account to display historical data. This means that the information will persist even if the dashboard is closed or reloaded.
 - From Device Bucket: This option is used to retrieve historical data specifically from a Device Bucket, which stores a history of device properties or aggregated data. It functions similarly to a Data Bucket but is intrinsically linked to a device's historical properties.
 - From Device Property: This option is ideal for retrieving data from a device's configurable properties, making it particularly useful for visualizing static or infrequently changing device configuration data, such as firmware versions or location IDs, as well as displaying the last received data from HTTP devices that do not maintain a persistent connection.

✓ When working with device data, it is recommended to use a product data bucket instead of individual data buckets. This way, just one rule will monitor every product devices.

Multiple data sources

In the "Data Sources" section of the rule configuration menu in Thingier.io, the **" + Add Source"** button allows users to add variables to a specific rule. This feature is essential for configuring the rule with the necessary data for evaluation.

Activation

The alarm activation refers to the process where the system monitors the variables specified in the "Sources" section and generates notifications when these conditions are met. This means the system continuously checks the data against the defined criteria to detect any deviations or specific events. It's also important to note that it is possible to define confirmation criteria that prevent false activations.

The alarm activation section in Thingier.io consists of a form with three main sections: **Conditions**, **Confirmation**, and **Notification**. Each of these sections plays a crucial role in defining how and when alarms are triggered and how notifications are managed.

The screenshot shows the 'Activation' tab of the Thingier.io configuration interface. It features four tabs at the top: 'Data Source', 'Activation' (selected), 'Normalization', and 'Reminder'. The 'Activation' section is divided into three main areas: 'Conditions', 'Confirmation', and 'Notification'. The 'Conditions' section has an information icon and a table with one row: 'When', 'Data1.temperature', 'IS ABOVE', and '25'. There is a '+ Add Condition' button below. The 'Confirmation' section has an information icon and a 'Timespan' dropdown set to '5' minutes. The 'Notification' section has an information icon, a 'Select Endpoint...' dropdown, and a '+ Add Notification' button.

i In this example, we are using the variable "temperature" which comes from a datapoint that contains multiple variables:

```
Data1
{
  "temperature":20,
  "humidity":50,
  "pressure":850
}
```

The desired variable can be selected through the following structure in the input text:
"Data1.temperature"

- **Conditions:** allows users to define one or more comparisons of a selected variable and a setpoint. These conditions specify the exact criteria that must be met for the alarm to be activated. By setting these conditions, users can precisely monitor the desired metrics and ensure alarms are only triggered when specific thresholds or ranges are met. The comparisons can be:
 - Greater than
 - Less than
 - Equal to
 - Not equal to
 - Within a range
 - Outside of a range



Note that it is possible to create more complex behaviors by adding additional conditions to the rule

- **Confirmation:** This section helps to avoid false activations of the alarm. It is an essential step for filtering out transient or spurious events that do not require immediate attention, thus ensuring that alarms are only generated for sustained or repeated conditions. This can be configured in several ways:
 - Immediate Confirmation: The alarm is triggered as soon as the condition is met.
 - Sequence: The alarm is activated if the condition is met multiple times
 - Timespan: The alarm is only triggered if the condition remains true for a specified number of consecutive valid comparisons.
- **Notification:** This section allows users to define the endpoint profile (previously defined) that will be used to send the notification if the alarm is necessary. By configuring the notification settings, users can ensure that the right people are informed promptly about critical conditions, facilitating a quick and appropriate response.

Understanding Alarm Activation Fields

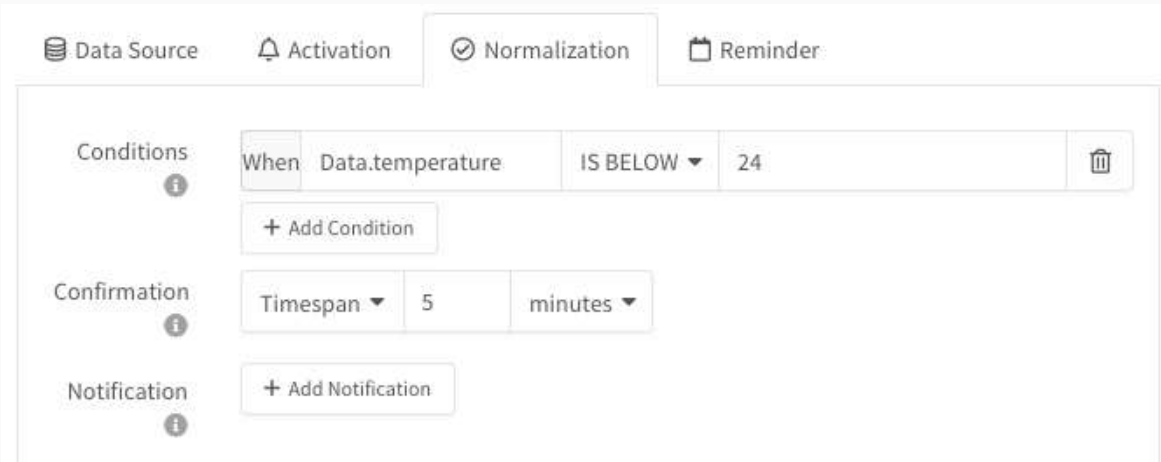
The following fields are available to determine when an alarm should be activated:

- **device:** The unique identifier of the device.
- **created:** Timestamp indicating when the device was originally created.
- **modified:** Timestamp of the last modification to the device's settings or status.
- **enabled:** Whether the device is currently enabled. Alarms are typically only triggered for devices that are enabled.
- **connection.ts:** The timestamp of the last known connection attempt or session.
- **connection.active:** This is the most critical field. It indicates whether the device is currently online and connected to the Thingier.io server. Use this field to detect connectivity issues or trigger alerts when a device goes offline.

These fields allow alarms to be precisely tuned to changes in device state, ensuring that only relevant alerts are generated.

Normalization

The normalization section of the rules configuration allows users to define the platform's behavior for deactivating an alarm when the necessary conditions for it to be cleared are met in the device's data. This configuration uses the same elements as in the activation section, but with the focus on setting values that ensure a reliable deactivation of the alarm.



The screenshot shows the 'Normalization' tab in a configuration interface. At the top, there are four tabs: 'Data Source', 'Activation', 'Normalization' (which is selected and highlighted with a checkmark), and 'Reminder'. Below the tabs, the configuration is organized into three sections: 'Conditions', 'Confirmation', and 'Notification'. Each section has an information icon (i) to its left. The 'Conditions' section contains a table with one row: 'When', 'Data.temperature', 'IS BELOW', and '24'. There is a trash icon to the right of the table. Below the table is a '+ Add Condition' button. The 'Confirmation' section contains a 'Timespan' dropdown menu set to '5' and a 'minutes' dropdown menu. The 'Notification' section contains a '+ Add Notification' button.

Conditions
When
Data.temperature
IS BELOW
24

+ Add Condition

Confirmation

Timespan 5 minutes

Notification

+ Add Notification

Reminders

The **Reminder** section of the alarm configuration is designed to provide ongoing notifications at regular intervals when an alarm remains active. This feature ensures that critical alarms are not overlooked or forgotten over time. It is possible to select two elements:

- **Interval:** Users can specify the time interval at which the reminders should be sent. This can be configured to match the urgency of the alarm, with shorter intervals for more critical issues and longer intervals for less urgent ones.
- **Notification:** Users can define the endpoint profile for the reminder notifications. This includes the communication method (e.g., email, SMS, push notifications) and the details for delivering these reminders.

Alarm list

The alarm list Thinger.io is a comprehensive interface that displays all the active alarms generated by the system when the specified conditions are met. It provides a detailed overview of all current alarms that have been triggered. This list helps users to track and manage ongoing issues across their IoT devices and systems.

Alarms > instances

InstancesRules

Refresh

Search Alarm

☐	Alarm	Origin	Severity	State	Values	Seen	Duration	Annotation	Project
☐	property C-518503195	ClimaStick1	Medium	Activated	myProp.value.software_version -169	Now	5d 21h 40m 58s		ExampleProject

Showing 1 alarm

Key Data Displayed in the Alarms List

- **Alarm:** Each alarm is assigned a unique ID for easy reference and tracking.
- **Origin:** It's the source of the alarm, it indicates the device or data source that triggered the alarm. This helps identify the exact point of origin of the alarm.

- **Severity:** Displays the severity of the alarm, as it was defined in the rule configuration. This feature is aimed at helping to prioritize response actions. Severity levels might include *Critical*, *High*, *Medium*, *Low* and *None*.
- **State:** The status of the Alarm, which shows the current state of the alarm and could be:
 - **Activated:** The alarm is currently active and has not been resolved.
 - **Latch:** The alarm has been acknowledged but remains active.
 - **Shelve:** The alarm has been temporarily suppressed or postponed.
- **Values:** Displays the trigger Variable, the specific variable that triggered the alarm, along with its current value, helping to understand the cause of the alarm.

1. Visualized Time

- **Timestamp:** Indicates the moment when the user first viewed the alarm. This helps in tracking how long the alarm has been known to the team.

2. Elapsed Time

- **Duration:** Shows the time elapsed since the alarm was activated. This information is critical for measuring response times and understanding the persistence of issues.

Alarm management

The alarm list also includes several control functions that appear when one or more alarm instances are selected. These controls allow users to manage the alarms effectively.

< 3 Acknowledge Shelve Latch Clear Set Projects Remove					
☐	Alarm	Origin	Severity	State	Values
☐	propety C-518503195	ClimaStick1	● Medium	 Activated	myProp.value.software_version -169
Showing 1 alarm					

Alarm Management in the Alarm List is a critical function that allows users to maintain control over active alarms, ensuring they are addressed appropriately and efficiently. Here's how each control contributes to effective alarm management:

- **Acknowledge:** When users acknowledge an alarm, it signifies that the alarm has been noticed and is being addressed. This step is essential in a team environment to prevent multiple people from unknowingly working on the same issue.
- **Shelve:** Shelving an alarm allows users to temporarily suppress it, reducing noise and distraction from alarms that are known but not immediately actionable. This helps in focusing on more urgent alarms while ensuring less critical ones are revisited later.
- **Latch:** Latching an alarm ensures it remains active until a manual action is taken to clear it. This is particularly useful for severe issues that require explicit confirmation of resolution, ensuring accountability and thoroughness in alarm management.
- **Clear:** Clearing an alarm is the final step once the issue has been resolved. This keeps the alarm list up-to-date and focused only on current, unresolved issues, helping users to quickly identify and respond to ongoing problems.
- **Set Projects:** Assigning alarms to projects helps organize and manage them within specific contexts. This is particularly useful in larger deployments where alarms may pertain to different devices, locations, or operational areas.
- **Remove:** Removing alarms from the system is necessary for cleaning up outdated or irrelevant alarms. This ensures the Alarm List remains relevant and uncluttered, making it easier to manage current alarms.

Alarms Inspector

The Alarms Inspector in Thinger.io is a tool designed to provide detailed insights into alarm behavior. It helps users track and analyze the alert system by displaying the event history of alarms. It aids in understanding the lifecycle and behavior of each alarm by providing a detailed timeline of events.

Filtering Options

The Alarms Inspector offers several filtering options, allowing users to **focus on specific alarm event types**. These filters include:

- **alarm_instance_activate:** Shows events where an alarm instance is activated.

- **alarm_instance_create:** Displays events where a new alarm instance is created.
- **alarm_instance_delete:** Shows events where an alarm instance is deleted.
- **alarm_instance_normalize:** Displays events where an alarm instance is normalized, indicating that the triggering condition is back to normal.
- **alarm_instance_update:** Shows events where an alarm instance is updated with new information or conditions.
- **alarm_rule_create:** Displays events where a new alarm rule is created.
- **alarm_rule_delete:** Shows events where an alarm rule is deleted.
- **alarm_rule_execute:** Displays events where an alarm rule is executed, showing the process of checking conditions and triggering alarms.
- **alarm_rule_update:** Shows events where an alarm rule is updated with new parameters or conditions.

Device-events

The platform provides access to a set of system-generated signals that represent the current **state and metadata** of each device. These signals can be used for monitoring, automation, and alarm generation.

The available fields include:

- **device:** The unique identifier (device ID).
- **created:** Timestamp indicating when the device was initially created.
- **modified:** Timestamp of the most recent configuration change.
- **enabled:** Boolean flag indicating whether the device is currently enabled or disabled.
- **connection.ts:** Timestamp of the last known connection activity.
- **connection.active:** Boolean indicating whether the device is currently connected.

These signals are particularly useful for building automation logic or triggering alerts based on device status. For instance, a common practice when defining alarms is to **verify that a device is both disconnected and enabled**, in order to avoid generating alerts for devices that are intentionally disabled (e.g., under maintenance or decommissioned).

By leveraging these signals, users can implement reliable and meaningful monitoring flows that reflect the real operational status of their IoT infrastructure.

Previous
ENDPOINTS

Next
ACCESS TOKENS

Last updated 4 months ago

Was this helpful?

